

AL1GN
(Application-Layer 1-on-1 Gaming Network)

จัดทำโดย
6710405320 ดุลย์ธานี ฟังเกียรติรัมย์

สารบัญ

1. บทนำ.....	4
2. วัตถุประสงค์และคำอธิบายโปรแกรม	5
2.1 วัตถุประสงค์.....	5
2.2 คำอธิบายโปรแกรม	5
2.3 ตัวอย่างเกมที่รองรับ.....	5
3. คุณลักษณะของแอปพลิเคชัน	6
3.1 เหตุผลที่ไม่ใช้โปรโตคอลมาตรฐาน	6
4. เหตุผลในการเลือกใช้ TCP.....	7
4.1 ต้องการความน่าเชื่อถือของข้อมูล (Reliable Data Transfer)	7
4.2 ต้องการลำดับการรับข้อมูล (Ordered Delivery)	7
4.3 รูปแบบโปรโตคอลเป็น Stream-based	7
4.4 สรุป	7
5. โมเดลการทำงาน	8
5.1 รูปแบบข้อความ (Message Format).....	8
6. ขั้นตอนการทำงานของโปรโตคอล (AL1GN Procedures)	10
6.1 การเปิดและปิดการเชื่อมต่อ	10
6.2 การลงทะเบียนผู้เล่นและ Session Token (Player Registration & Auth)	10
6.2.1 การลงทะเบียนครั้งแรก.....	10
6.2.2 การ Reconnect ด้วย Token	10
6.2.3 กรณีผิดพลาด	11
6.3 การจับคู่ผู้เล่น (Matchmaking).....	11
6.3.1 Queue-based (อัตโนมัติ)	11
6.3.2 Room-based (ใช้รหัสห้อง).....	11
6.4 การดำเนินเกม (Game Session).....	12
6.4.1 การส่งการเดิน	12
6.4.2 รูปแบบ board_state.....	12
6.4.3 เงื่อนไขสิ้นสุดเกม.....	13
6.4.4 การตรวจสอบการเดินที่ไม่ถูกต้อง	13
6.5 การตรวจสอบการเชื่อมต่อ (Heartbeat)	14
6.6 การจัดการผู้เล่นตัดการเชื่อมต่อ (Disconnection Handling)	14
6.7 การขอเล่นซ้ำ (Rematch Negotiation).....	15

7. ข้อกำหนดของโปรโตคอล (AL1GN Specifications)	16
7.1 คำสั่ง (Commands).....	16
7.2 รหัสการตอบกลับ (Reply Codes)	16
7.3 ลำดับคำสั่งและการตอบกลับ (Command-Reply Sequences)	19
7.4 State Diagram	20
8. ตัวอย่างสถานการณ์ (Scenarios).....	21
สถานการณ์ที่ 1: การเชื่อมต่อและลงทะเบียน	21
สถานการณ์ที่ 2: เกม TTT ผ่านคิวิ (Alice ชนะ)	21
สถานการณ์ที่ 3: สร้างห้องและ Rematch.....	22
สถานการณ์ที่ 4: การตัดการเชื่อมต่อและ Reconnect ด้วย Token.....	23
สถานการณ์ที่ 5: ข้อผิดพลาดต่าง ๆ.....	23
9. พารามิเตอร์ที่ปรับแต่งได้ (Configurable Parameters)	24

1. บทนำ

โปรโตคอล AL1GN (Application-Layer 1-on-1 Gaming Network) เวอร์ชัน 1.0 เป็นโปรโตคอลระดับ Application Layer ที่ออกแบบมาเพื่อรองรับการสื่อสารในเกม 1 ต่อ 1 แบบผลัดกันเดิน (turn-based) ผ่านเครือข่ายคอมพิวเตอร์ โปรโตคอลนี้ทำงานบนพื้นฐานของ TCP socket และใช้รูปแบบข้อความแบบ text-based ที่อ่านได้ด้วยมนุษย์

AL1GN ออกแบบให้เป็น **โปรโตคอลทั่วไป (generalized protocol)** กล่าวคือ ตัวโปรโตคอลเองไม่ผูกติดกับเกมใดเกมหนึ่ง แต่รองรับเกม turn-based ได้หลายประเภทผ่าน payload ของข้อความ ในการนำไปใช้งานครั้งนี้ได้เลือกสาธิตด้วยเกม Tic-Tac-Toe (TTT) และ Connect4 (C4)

โปรโตคอลนี้ไม่ใช่ HTTP, WebSocket หรือโปรโตคอลมาตรฐานอื่น ๆ เนื่องจากแอปพลิเคชันเกมแบบ turn-based มีความต้องการเฉพาะที่โปรโตคอลเหล่านั้นไม่ตอบสนองได้อย่างมีประสิทธิภาพ ดังอธิบายในหัวข้อ 4

2. วัตถุประสงค์และคำอธิบายโปรแกรม

2.1 วัตถุประสงค์

โปรแกรมนี้พัฒนาเพื่อให้ผู้เล่นสองคนสามารถเล่นเกมกระดานแบบ turn-based ผ่านเครือข่ายได้ โดยมีเซิร์ฟเวอร์กลางทำหน้าที่เป็น **ผู้ตัดสิน (referee)** และ **ผู้จัดการสถานะเกม (authoritative state manager)** ผู้เล่นแต่ละคนใช้โปรแกรม client เชื่อมต่อเข้าสู่เซิร์ฟเวอร์ผ่าน TCP socket

2.2 คำอธิบายโปรแกรม

- **Server:** รับการเชื่อมต่อจาก client หลายตัวพร้อมกัน จัดการ session, จับคู่ผู้เล่น, ตรวจสอบความถูกต้องของการเดิน, และส่งสถานะเกมให้ผู้เล่นทั้งสองฝ่าย
- **Client:** ส่งคำสั่งไปยังเซิร์ฟเวอร์ (ลงทะเบียน, เข้าร่วมคิว, ส่งการเดิน) รับการอัปเดตสถานะเกม และแสดงกระดานเกมให้ผู้ใช้เห็น

2.3 ตัวอย่างเกมที่รองรับ

รหัสเกม	ชื่อเกม	ขนาดกระดาน	เงื่อนไขชนะ
TTT	Tic-Tac-Toe	3×3	เรียง 3 ในแนวตรง/ทแยง
C4	Connect4	6×7	เรียง 4 ในแนวตรง/ทแยง

3. คุณลักษณะของแอปพลิเคชัน

คุณลักษณะ	รายละเอียด
สถาปัตยกรรม	Client-Server
การเชื่อมต่อ	Persistent connection (เชื่อมต่อครั้งเดียว เล่นได้หลายเกม)
สถานะ	Stateful; เซิร์ฟเวอร์เก็บสถานะเกมตลอดเวลา
การสื่อสาร	Bidirectional; server สามารถ push ข้อความไปหา client ได้โดยไม่ต้องรอคำขอ
ผู้ใช้งาน	2 ผู้เล่น (1v1) ต่อเกม
การตรวจสอบสิทธิ์	ชื่อผู้เล่น + session token (ออกโดย server)
รูปแบบข้อความ	Text-based, human-readable, ASCII
ตัวค้นข้อความ	\r\n (CRLF) ต่อท้ายทุกข้อความ

3.1 เหตุผลที่ไม่ใช้โปรโตคอลมาตรฐาน

- **ไม่ใช่ HTTP:** HTTP เป็น stateless protocol ที่ใช้รูปแบบ request-response แบบ client-pull การรับการแจ้งเตือนแบบ real-time จำเป็นต้องใช้ polling ซึ่งสิ้นเปลืองทรัพยากรและมี latency สูง
- **ไม่ใช่ WebSocket:** WebSocket ต้องการ HTTP handshake เบื้องต้นและ masking overhead ที่ไม่จำเป็นสำหรับแอปพลิเคชัน native socket
- **ALIGN:** ออกแบบมาเฉพาะสำหรับ turn-based game มี state machine ที่ชัดเจน รหัสสถานะที่ตรงต่อความหมายของเกม และรองรับ server push โดยตรง

4. เหตุผลในการเลือกใช้ TCP

AL1GN เลือกใช้ TCP (Transmission Control Protocol) เป็น Transport Layer service model ด้วยเหตุผลดังต่อไปนี้

4.1 ต้องการความน่าเชื่อถือของข้อมูล (Reliable Data Transfer)

เกมกระดาน turn-based มีสถานะที่ **สอดคล้อง**เนื่อง

กล่าวคือการเดินแต่ละครั้งเปลี่ยนแปลงสถานะกระดานอย่างถาวร หากแพ็กเก็ตที่บรรจุคำสั่ง MOVE สูญหาย เซิร์ฟเวอร์และ client จะมีสถานะกระดานที่ **ไม่ตรงกัน (desynchronized)** อย่างถาวร ซึ่ง UDP ไม่รับประกันการส่งข้อมูลครบถ้วน

4.2 ต้องการลำดับการรับข้อมูล (Ordered Delivery)

ลำดับของการเดินมีความสำคัญอย่างยิ่ง เช่น MOVE A -> MOVE B ต้องถูกประมวลผลตามลำดับนั้นเสมอ TCP รับประกันว่าข้อมูลถึงปลายทางตามลำดับที่ส่ง (in-order delivery)

4.3 รูปแบบโปรโตคอลเป็น Stream-based

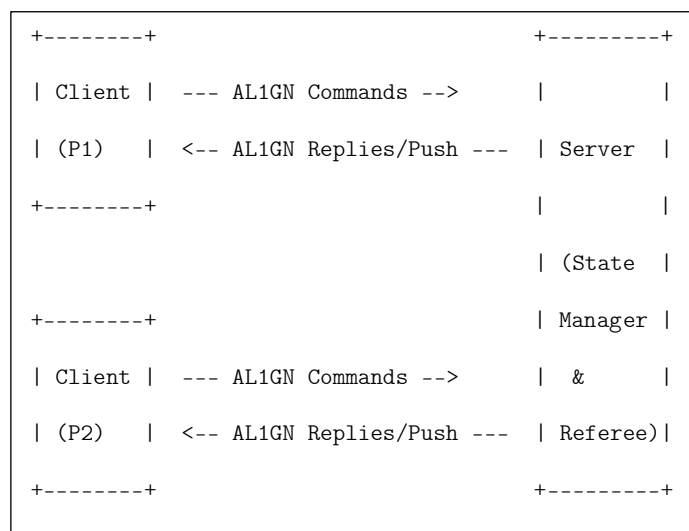
AL1GN ใช้ข้อความ text-based ที่คั่นด้วย CRLF โดยอาศัย byte stream ที่ต่อเนื่องของ TCP ในการแยกข้อความออกจากกัน ซึ่งเหมาะกับ TCP stream socket โดยธรรมชาติ

4.4 สรุป

คุณสมบัติ	TCP	UDP	ความจำเป็นสำหรับ AL1GN
Reliable delivery	✓	X	จำเป็น: การเดินห้ามหาย
Ordered delivery	✓	X	จำเป็น: ลำดับการเดินสำคัญ
Error detection	✓	บางส่วน	จำเป็น
Connection-oriented	✓	X	จำเป็น: session management
Low overhead	X	✓	ไม่จำเป็น: game events ไม่ได้ส่งบ่อยมาก

5. โมเดลการทำงาน

AL1GN ใช้โมเดล **Client-Server** โดยมีหลักการดังนี้



- Client ส่ง คำสั่ง (Command) ไปยัง Server
- Server ตอบกลับด้วย Reply ต่อทุกคำสั่งที่ได้รับ
- Server สามารถส่ง Push Message ไปยัง client ที่เป็นคู่ต่อสู้ได้โดยไม่ต้องรอคำสั่ง (เช่น แจ้งว่าถึงตาเดิน)
- Server คือผู้ตัดสินที่มีอำนาจสูงสุด (authoritative); ผลการตรวจสอบการเดินของ server ถือเป็นที่สุด

5.1 รูปแบบข้อความ (Message Format)

คำสั่งจาก Client:

<COMMAND> [<SP> <argument>] <CRLF>

การตอบกลับจาก Server:

<3-digit-code> <SP> <phrase> [<SP> <data>] <CRLF>

ตัวอย่าง:

C: HELO PlayerOne\r\n

S: 201 Session_created\r\n

C: MOVE 1,2\r\n

S: 204 Move_accepted\r\n

S: 301 Game_turn PlayerTwo X|O|.\r\n <-- push ไปยังคู่ต่อสู้

- <COMMAND>: ตัวอักษรพิมพ์ใหญ่ 4-7 ตัว
- <3-digit-code>: รหัสสถานะ 3 หลัก
- <SP>:ช่องว่าง (ASCII 32)
- <CRLF>: \r\n (ASCII 13, 10)
- คำสั่งและรหัสไม่คำนึงถึงตัวพิมพ์เล็ก/ใหญ่ (case-insensitive) ยกเว้น argument ที่เป็น player name และ room code

6. ขั้นตอนการทำงานของโปรโตคอล (AL1GN Procedures)

6.1 การเปิดและปิดการเชื่อมต่อ

เมื่อ client เชื่อมต่อ TCP สำเร็จ เซิร์ฟเวอร์จะส่งข้อความต้อนรับทันที:

S: 220 Service_ready\r\n

การปิดการเชื่อมต่ออย่างปกติ:

C: QUIT\r\n

S: 221 Service_closing_connection\r\n

หลังจากนั้น server จะปิด TCP socket

6.2 การลงทะเบียนผู้เล่นและ Session Token (Player Registration & Auth)

คำสั่งแรกที่ client ต้องส่งหลังได้รับ 220 คือ HELO พร้อมชื่อผู้เล่น

6.2.1 การลงทะเบียนครั้งแรก

C: HELO PlayerOne\r\n

S: 201 Session_created <token>\r\n

Server จะออก **session token** (สตริง hex แบบสุ่ม 16 ตัวอักษร) พร้อมกับการยืนยันการลงทะเบียน
Client ควรเก็บ token นี้ไว้เพื่อใช้ reconnect

6.2.2 การ Reconnect ด้วย Token

หาก client ตัดการเชื่อมต่อและต้องการกลับเข้าเกมเดิม ให้ส่ง HELO พร้อมทั้งชื่อและ token:

C: HELO PlayerOne <token>\r\n

S: 207 Session_restored <board_state>\r\n

Token ป้องกันไม่ให้ผู้อื่นแอบอ้างชื่อเพื่อเข้าร่วมเกมที่กำลังดำเนินอยู่

6.2.3 กรณีผิดพลาด

สถานการณ์	Server ตอบกลับ
ชื่อนี้มีผู้ใช้งานอยู่แล้ว (live session)	433 Name_already_taken
ชื่อตรง แต่ token ไม่ตรง	460 Bad_token
ชื่อตรง และ token ตรง	207 Session_restored ...

6.3 การจับคู่ผู้เล่น (Matchmaking)

AL1GN รองรับการจัดคู่ 2 วิธี:

6.3.1 Queue-based (อัตโนมัติ)

C: QUEUE TTT\r\n

S: 300 Waiting_for_opponent\r\n

เมื่อมีผู้เล่นอีกคนเข้าคิวเกมชนิดเดียวกัน server จะจับคู่ทั้งสองและ broadcast

ข้อความเดียวกันไปยังผู้เล่นทั้งคู่:

S: 301 Game_turn <active_player_name> <board_state>\r\n

<active_player_name> คือชื่อของผู้เล่นที่ถึงตาเดิน แต่ละ client

เปรียบเทียบชื่อนี้กับชื่อของตัวเองเพื่อทราบว่าถึงตาตัวเองหรือไม่

6.3.2 Room-based (ใช้รหัสห้อง)

ผู้เล่นที่ต้องการเล่นกับคนที่รู้จัก:

-- ผู้สร้างห้อง --

C: MAKE TTT\r\n

S: 202 Room_created ABC123\r\n

-- ผู้เข้าร่วม --

C: JOIN ABC123\r\n

S: 203 Joined_room ABC123 TTT\r\n

S->P1: 301 Game_turn <active_player_name> <board_state>\r\n

S->P2: 301 Game_turn <active_player_name> <board_state>\r\n

6.4 การดำเนินเกม (Game Session)

6.4.1 การส่งการเดิน

Tic-Tac-Toe: argument คือ `row,col` (0-indexed, 0-2)

C: MOVE 1,1\r\n

S: 204 Move_accepted\r\n

Connect4: argument คือ `col` (0-indexed, 0-6)

C: MOVE 3\r\n

S: 204 Move_accepted\r\n

หลังจากตอบกลับ 204 server จะ broadcast **301 Game_turn** ไปยังผู้เล่นทั้งคู่:

S->both: 301 Game_turn <next_player_name> <board_state>\r\n

6.4.2 รูปแบบ board_state

`board_state` คือ string ที่แสดงสถานะกระดานในบรรทัดเดียว โปรโตคอลเองไม่ได้เป็นผู้กำหนด ขึ้นอยู่กับผู้นำโปรโตคอลไปใช้งาน

ณ ที่นี้จะยกตัวอย่างโดยใช้ | คั่นระหว่างช่อง และ / คั่นระหว่างแถว สัญลักษณ์: X, O, . (ว่าง)

ตัวอย่าง TTT หลังเดิน 2 ครั้ง:

301 Your turn X,.,.|.,0,.,|.,.,.\r\n

ตัวอย่าง Connect4 (แสดง 6 แถว × 7 คอลัมน์):

301 Your turn .,.,.,.,.,.,./.,.,.,.,.,.,./.,.,.,.,.,.,./
.,.,.,.,.,.,./.,.,.,.,.,.,./.,.,.,X,.,.,.\r\n

6.4.3 เงื่อนไขสิ้นสุดเกม

เมื่อเกมจบ server จะส่ง push message ไปยัง ผู้เล่นทั้งสอง พร้อม board_state สุดท้าย:

สถานการณ์	ข้อความที่ผู้เล่นได้รับ	ข้อความที่ผู้แพ้ได้รับ
มีผู้ชนะ	303 Game_over_Win [board]	304 Game_over_Loss [board]
เสมอ	305 Game_over_Draw [board]	305 Game_over_Draw [board]
คู่ต่อสู้ตัดการเชื่อมต่อ	306 Game_over_Forfeit	-

6.4.4 การตรวจสอบการเดินที่ไม่ถูกต้อง

AL1GN เป็น **generalized protocol** ที่ไม่ผูกติดกับเกมใดเกมหนึ่ง โปรโตคอลกำหนดเพียงว่า:

- การเดินที่ server ยอมรับจะได้รับ 204 Move_accepted
- การเดินที่ server ปฏิเสธจะได้รับ 451 Invalid_move

"การเดินที่ถูกต้อง" ไม่ใช่ข้อกำหนดของโปรโตคอล แต่เป็นความรับผิดชอบของ board implementation ที่ protocol adopter จัดเตรียมมา ตัวอย่างเช่น:

- Tic-Tac-Toe: ช่องต้องว่าง, อยู่ในกระดาน 3x3
- Connect4: คอลัมน์ต้องไม่เต็ม, อยู่ใน 0-6
- เกมอื่น ๆ ในอนาคต: กำหนดกฎเองทั้งหมด

ผลดีคือ server ไม่จำเป็นต้องรู้กฎของเกม ระบบจัดการเกมเพียงเรียกเมธอดตรวจสอบกฎ และตอบกลับตามผลลัพธ์

C: MOVE 1,1\r\n <-- ช่องที่มีหมากแล้ว (ตามกฎ TTT)

S: 451 Invalid_move\r\n

C: MOVE 1,2\r\n <-- ไม่ใช่ตาของผู้เล่นคนนี้

S: 450 Not_your_turn\r\n

6.5 การตรวจสอบการเชื่อมต่อ (Heartbeat)

Server จะส่ง PING ไปยัง client ทุก ๆ HEARTBEAT_INTERVAL วินาที (ค่า default: 30 วินาที)

S: PING\r\n

C: PONG\r\n

S: 206 Pong_received\r\n

หาก client ไม่ตอบ PONG ภายใน PING_TIMEOUT วินาที server จะถือว่า client ตัดการเชื่อมต่อ

6.6 การจัดการผู้เล่นตัดการเชื่อมต่อ (Disconnection Handling)

เมื่อ server ตรวจพบว่าผู้เล่นตัดการเชื่อมต่อระหว่างเกม:

1. Server แจ้งผู้เล่นที่ยังออนไลน์อยู่:S: 308 Opponent_disconnected Waiting_for_reconnect\r\n

2. Server เริ่มนับถอยหลัง RECONNECT_TIMEOUT วินาที (ค่า default: 60 วินาที)

3. หากผู้เล่นที่ตัดการเชื่อมต่อกลับมาเชื่อมต่อและส่ง HELO <same_name> <token> ภายใน timeout:C: HELO PlayerOne <token>\r\n

S: 207 Session_restored [board_state]\r\nเกมดำเนินต่อจากจุดที่ค้างไว้

4. หาก timeout หมดอายุก่อนที่จะมีการ reconnect:S->remaining_player: 306
Game_over_Forfeit\r\n

6.7 การขอเล่นซ้ำ (Rematch Negotiation)

หลังจากเกมจบ ผู้เล่นทั้งสองอยู่ในสถานะ PostGame และสามารถขอเล่นซ้ำได้:

-- ผู้เล่นที่ต้องการเล่นซ้ำ --

C: REMATCH\r\n

S: 200 OK\r\n

-- server แจ้งคู่ต่อสู้ --

S->opponent: 307 Rematch_requested\r\n

-- คู่ต่อสู้ตอบรับ --

C: ACCEPT\r\n

S: 205 Rematch_accepted\r\n

-- server broadcast เกมใหม่ให้ทั้งคู่ --

S->both: 301 Game_turn <active_player_name> <board_state>\r\n

หากคู่ต่อสู้ปฏิเสธ:

C: DECLINE\r\n

S: 200 OK\r\n

S->requester: 309 Rematch_declined\r\n

7. ข้อกำหนดของโปรโตคอล (AL1GN Specifications)

7.1 คำสั่ง (Commands)

คำสั่งทั้งหมดต้องลงท้ายด้วย <CRLF> (\r\n)

คำสั่ง	ทิศทาง	รูปแบบ	คำอธิบาย
HELO	C->S	HELO <name>\r\n	ลงทะเบียนครั้งแรก — server ออก token
HELO	C->S	HELO <name> <token>\r\n	Reconnect — ยืนยันตัวตนด้วย token
QUEUE	C->S	QUEUE <game>\r\n	เข้าร่วมคิวจับคู่อัตโนมัติ (ตัวอย่าง game: TTT หรือ C4)
MAKE	C->S	MAKE <game>\r\n	สร้างห้องส่วนตัว server ตอบด้วยรหัสห้อง
JOIN	C->S	JOIN <code>\r\n	เข้าร่วมห้องด้วยรหัส
MOVE	C->S	MOVE <position>\r\n	ส่งการเดิน (TTT: row,col / C4: col)
PING	S->C	PING\r\n	Heartbeat จาก server
PONG	C->S	PONG\r\n	ตอบกลับ Heartbeat
REMATCH	C->S	REMATCH\r\n	ขอเล่นเกมใหม่หลังเกมจบ
ACCEPT	C->S	ACCEPT\r\n	ยอมรับคำขอ rematch
DECLINE	C->S	DECLINE\r\n	ปฏิเสธคำขอ rematch
QUIT	C->S	QUIT\r\n	ตัดการเชื่อมต่ออย่างปกติ

7.2 รหัสการตอบกลับ (Reply Codes)

รหัสการตอบกลับเป็นตัวเลข 3 หลัก ตามด้วยช่องว่างและข้อความอธิบาย กลุ่มของรหัสมีดังนี้:

- 2xx — สำเร็จ (Success)
- 3xx — สถานะเกม (Game State)
- 4xx — ข้อผิดพลาดจาก client (Client Error)
- 5xx — ข้อผิดพลาดจาก server (Server Error)

2xx — สำเร็จ

รหัส	Phrase	ความหมาย
200	OK	คำสั่งสำเร็จทั่วไป
201	Session_created <token>	ลงทะเบียนสำเร็จ — token แบนมาด้วย
202	Room_created <code>	สร้างห้องสำเร็จ — รหัสห้องแนบมาด้วย
203	Joined_room <code> <game>	เข้าร่วมห้องสำเร็จ
204	Move_accepted	server ยืนยันการเดิน
205	Rematch_accepted	เริ่มเกมใหม่
206	Pong_received	ตอบ heartbeat
220	Service_ready	พร้อมให้บริการ (ส่งเมื่อเชื่อมต่อสำเร็จ)
221	Service_closing_connection	ปิดการเชื่อมต่ออย่างปกติ

3xx — สถานะเกม (Game State)

รหัส	Phrase	ความหมาย
300	Waiting_for_opponent	รอคู่ต่อสู้เข้าร่วม
301	Game_turn <active_name> <board>	Broadcast ไปยัง ผู้เล่นทั้งคู่ — <active_name> คือชื่อผู้ที่ถึงตาเดิน
303	Game_over_Win <board>	คุณชนะ พร้อม board_state สุดท้าย
304	Game_over_Loss <board>	คุณแพ้ พร้อม board_state สุดท้าย
305	Game_over_Draw <board>	เสมอ พร้อม board_state สุดท้าย
306	Game_over_Forget	คู่ต่อสู้ตัดการเชื่อมต่อและหมด timeout
307	Rematch_requested	คู่ต่อสู้ขอเล่นซ้ำ
308	Opponent_disconnected	คู่ต่อสู้ตัดการเชื่อมต่อ กำลังรอ reconnect
309	Rematch_declined	คู่ต่อสู้ปฏิเสธการเล่นซ้ำ

4xx — ข้อผิดพลาดจาก Client

รหัส	Phrase	ความหมาย
400	Bad_request	รูปแบบข้อความไม่ถูกต้อง
430	Not_registered	ยังไม่ได้ส่ง HELO ก่อน
431	Room_not_found	ไม่พบรหัสห้องที่ระบุ
432	Room_full	ห้องมีผู้เล่นครบ 2 คนแล้ว
433	Name_already_taken	ชื่อผู้เล่นนี้ถูกใช้ไปแล้ว
434	Unknown_game_type	ระบุชนิดเกมไม่ถูกต้อง
450	Not_your_turn	ส่ง MOVE ในตาของคู่ต่อสู้
451	Invalid_move	การเดินผิดกฎเกม (game-defined)
452	Game_not_started	ส่ง MOVE ก่อนเกมเริ่ม
460	Bad_token	Token ไม่ตรงกับชื่อที่ระบุ

5xx — ข้อผิดพลาดจาก Server

รหัส	Phrase	ความหมาย
500	Syntax_error	ไม่รู้จักคำสั่ง
501	Bad_parameter	argument ของคำสั่งไม่ถูกต้อง
503	Bad_command_sequence	ลำดับคำสั่งไม่ถูกต้อง
520	Internal_server_error	ข้อผิดพลาดภายใน server
521	Service_closing	server กำลังปิดให้บริการ

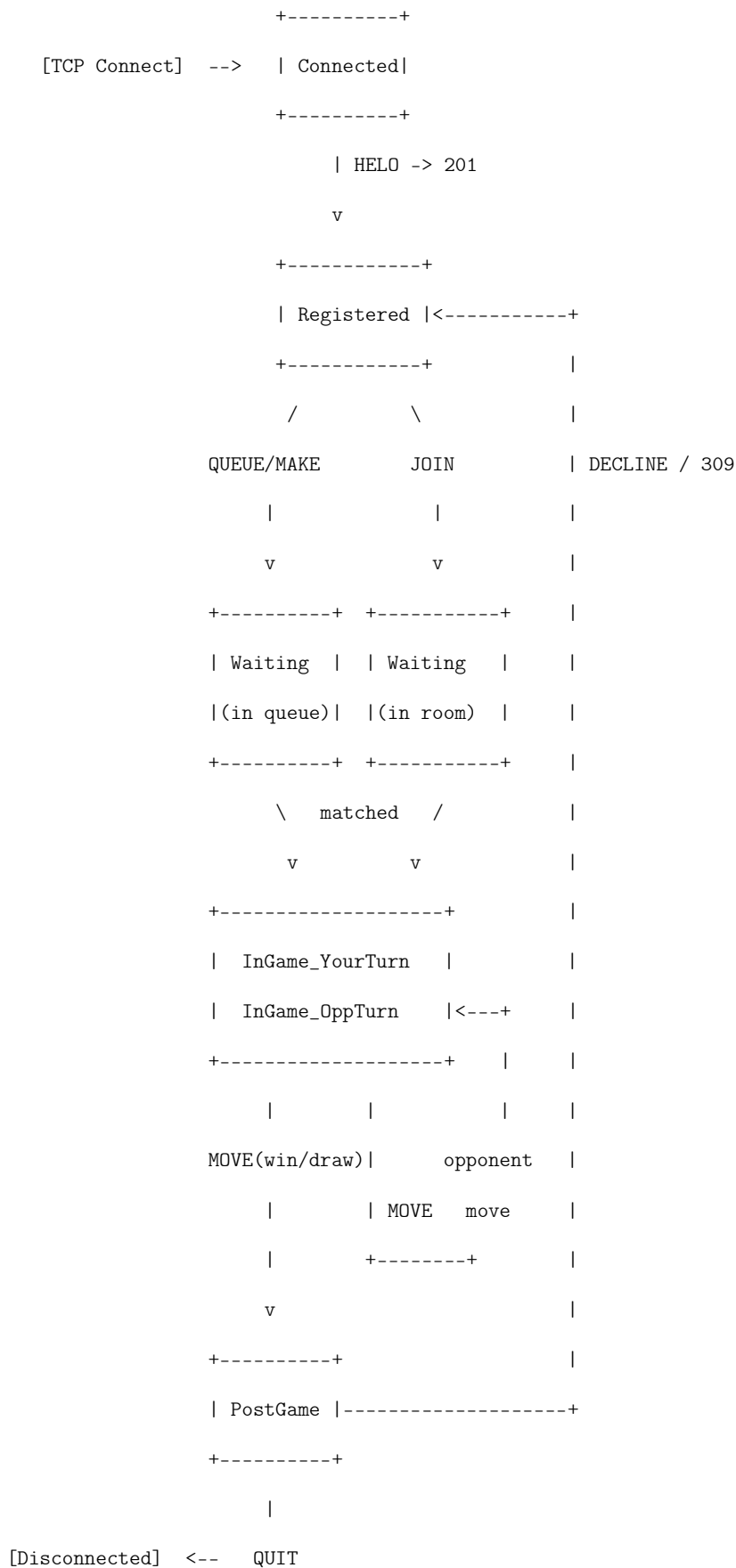
7.3 ลำดับคำสั่งและการตอบกลับ (Command-Reply Sequences)

คำสั่ง	ตอบกลับเมื่อสำเร็จ	ตอบกลับเมื่อผิดพลาด
HELO (ใหม่)	201 + token	433, 501
HELO (reconnect)	207 restored	433, 460, 501
QUEUE	300	430, 434, 503
MAKE	202	430, 434, 503
JOIN	203	430, 431, 432, 503
MOVE	204	430, 450, 451, 452, 501
PONG	206	—
REMATCH	200	430, 503
ACCEPT	205	430, 503
DECLINE	200	430, 503
QUIT	221	—

กฎลำดับคำสั่ง:

1. คำสั่งแรกต้องเป็น HELO เสมอ ยกเว้น QUIT ที่ส่งได้ตลอดเวลา
2. QUEUE, MAKE, JOIN ต้องส่งหลัง HELO เท่านั้น
3. MOVE ต้องส่งในระหว่างเกม (สถานะ InGame) และเฉพาะตาของตัวเอง
4. REMATCH, ACCEPT, DECLINE ต้องส่งหลังเกมจบ (สถานะ PostGame)
5. PONG ต้องส่งหลังได้รับ PING เท่านั้น
6. คำสั่งที่ส่งผิดลำดับจะได้รับ 503 Bad_command_sequence

7.4 State Diagram



8. ตัวอย่างสถานการณ์ (Scenarios)

สถานการณ์ที่ 1: การเชื่อมต่อและลงทะเบียน

```
S: 220 Service_ready \r\n

C: HELO Alice\r\n

S: 201 Session_created a3f8c2d1e9b047ab\r\n
```

สถานการณ์ที่ 2: เกม TTT ผ่านคิว (Alice ชนะ)

```
-- Alice (Client 1) --                -- Bob (Client 2) --

C: QUEUE TTT\r\n                      C: QUEUE TTT\r\n

S: 300 Waiting_for_opponent\r\n        S: 300 Waiting_for_opponent\r\n

-- server จับคู่สำเร็จ - broadcast เดียวกันทั้งคู่ --

S: 301 Game_turn Alice .,.,./.,.,./.,.,.\r\n
                                         S: 301 Game_turn Alice .,.,./.,.,./.,.,.\r\n

-- Alice เห็นชื่อตัวเอง -> ถึงตาฉัน --

-- Bob เห็นชื่อ Alice -> รอ --

C: MOVE 0,0\r\n

S: 204 Move_accepted\r\n

S: 301 Game_turn Bob X,.,./.,.,./.,.,.\r\n
                                         S: 301 Game_turn Bob X,.,./.,.,./.,.,.\r\n

                                         C: MOVE 1,1\r\n

                                         S: 204 Move_accepted\r\n

S: 301 Game_turn Alice X,.,./.,0,./.,.,.\r\n
                                         S: 301 Game_turn Alice X,.,./.,0,./.,.,.\r\n

C: MOVE 0,1\r\n

S: 204 Move_accepted\r\n

S: 301 Game_turn Bob X,X,./.,0,./.,.,.\r\n
                                         S: 301 Game_turn Bob X,X,./.,0,./.,.,.\r\n

                                         C: MOVE 2,2\r\n

                                         S: 204 Move_accepted\r\n

S: 301 Game_turn Alice X,X,./.,0,./.,.,0\r\n
                                         S: 301 Game_turn Alice X,X,./.,0,./.,.,0\r\n
```

```

C: MOVE 0,2\r\n
S: 204 Move_accepted\r\n
S: 303 Game_over_Win X,X,X/.,0,./.,.,0\r\n
S: 304 Game_over_Loss X,X,X/.,0,./.,.,0\r\n

```

สถานการณ์ที่ 3: สร้างห้องและ Rematch

```

-- Alice --
C: MAKE C4\r\n
S: 202 Room_created XK9P2M\r\n
S: 301 Game_turn Alice [board]\r\n
[... เกมดำเนินไป ...]
S: 304 Game_over_Loss [board]\r\n
C: REMATCH\r\n
S: 200 OK\r\n
S: 205 Rematch_accepted\r\n
S: 301 Game_turn Alice [board]\r\n

-- Bob --
C: JOIN XK9P2M\r\n
S: 203 Joined_room XK9P2M C4\r\n
S: 301 Game_turn Alice [board]\r\n
S: 303 Game_over_Win [board]\r\n
S: 307 Rematch_requested\r\n
C: ACCEPT\r\n
S: 205 Rematch_accepted\r\n
S: 301 Game_turn Alice [board]\r\n

```

สถานการณ์ที่ 4: การตัดการเชื่อมต่อและ Reconnect ด้วย Token

```
-- Alice --                                -- Bob --

[กำลังเล่นเกม...]

[Bob ตัดการเชื่อมต่อ]

S: 308 Opponent_disconnected Waiting_for_reconnect\r\n

[Bob reconnect และส่ง HELO พร้อม token]

C: HELO Bob b7d4e2f1a0c39815\r\n

S: 207 Session_restored [board_state]\r\n

S: 301 Game_turn Alice [board_state]\r\n

S: 301 Game_turn Alice [board_state]\r\n

[เกมดำเนินต่อ]
```

สถานการณ์ที่ 5: ข้อผิดพลาดต่าง ๆ

```
-- ส่ง MOVE ก่อน HELO --

C: MOVE 1,1\r\n

S: 430 Not_registered\r\n

-- ส่ง MOVE ผิดตา --

C: MOVE 2,2\r\n

S: 450 Not_your_turn\r\n

-- เดินในช่องที่มีหมากแล้ว (ตามกฎหมายของ TTT) --

C: MOVE 0,0\r\n

S: 451 Invalid_move\r\n

-- รหัสห้องไม่มี --

C: JOIN ZZZZZ\r\n

S: 431 Room_not_found\r\n

-- Reconnect ด้วย token ผิด --

C: HELO Bob wrongtoken123456\r\n

S: 460 Bad_token\r\n
```

9. พารามิเตอร์ที่ปรับแต่งได้ (Configurable Parameters)

ผู้ที่นำ AL1GN ไปใช้งาน (protocol adopter)

สามารถปรับค่าพารามิเตอร์ต่อไปนี้ได้ตามความเหมาะสมของสภาพแวดล้อม:

พารามิเตอร์	ค่า Default	คำอธิบาย
TURN_TIMEOUT	60 วินาที	เวลาสูงสุดที่ผู้เล่นมีในการส่ง MOVE หากเกินกำหนดถือว่าแพ้
RECONNECT_TIMEOUT	60 วินาที	เวลารอ reconnect ก่อนตัดสินใจให้ผู้เล่นที่ตัดการเชื่อมต่อแพ้
HEARTBEAT_INTERVAL	30 วินาที	ความถี่ในการส่ง PING จาก server
PING_TIMEOUT	10 วินาที	เวลารอ PONG ก่อนถือว่าตัดการเชื่อมต่อ
MAX_NAME_LENGTH	32 ตัวอักษร	ความยาวสูงสุดของชื่อผู้เล่น
ROOM_CODE_LENGTH	6 ตัวอักษร	ความยาวของรหัสห้อง
TOKEN_LENGTH	16 ตัวอักษร (hex)	ความยาวของ session token ที่สุ่มออกโดย server